ML Unit 2

Machine Learning — Unit 2 Notes

Tuning Complexity & Regularized Discriminant Analysis

- There is a **trade-off** between simplicity of a model and its generalization ability.
- When assumptions about the covariance matrix are made to reduce parameters, **bias is introduced**.

Regularized Discriminant Analysis (RDA)

In parametric classification with Gaussian densities, the covariance matrix can be written as a **weighted combination** of three special cases:

$$\hat{\Sigma} = \alpha \cdot \frac{1}{d}\text{tr}(\Sigma) \cdot I + (1-\alpha)\left[\beta \cdot \hat{\Sigma}_{pooled} + (1-\beta)\hat{\Sigma}_i\right]$$

α	β	**Result**
0	0	**Quadratic Classifier**
0	1	**Linear Classifier**
1	0	**Nearest Mean Classifier**

Linear Discriminant

A **linear discriminant** is a linear decision function used to separate data belonging to different classes using a straight line (2D), plane (3D), or hyperplane (higher dimensions).

Mathematical Form

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$$

Where:

- $\mathbf{x}$ = feature vector
- $\mathbf{w}$ = weight vector
- w_0 = bias (threshold)

Decision Rule

$$\text{Class} = \{\omega_1, \quad g(\mathbf{x}) \geq 0 \; \omega_2, \quad g(\mathbf{x}) < 0$$

Geometric Interpretation

Dimensions	Decision Boundary
2D	Straight line
3D	Plane
nD	Hyperplane

Discrete Features

- In some applications, attributes take one of n different discrete values.
  - Example: `color ∈ {red, green, blue}` or `pixel ∈ {0, 1}`

Discriminant Function for Discrete Features

$$g_i(\mathbf{x}) = \log P(C_i) + \sum_j \log p_{ij}(x_j)$$

This is **linear** in the log-probability space.

Bag of Words (Document Categorization)

- Used in **document categorization** (e.g., classifying news reports).
- A priori d words are chosen.
- After training, $\hat{p}_{ij}$ estimates the probability that word j occurs in a document of type i.

Multinomial Case

- p_{ijk} = probability that attribute x_j belonging to class C_i takes value v_k

$$\sum_k p_{ijk} = 1$$

- If all attributes are **independent**:

$$p(\mathbf{x}|C_i) = \prod_j p_{ij}(x_j)$$

- Discriminant function:

$$g_i(\mathbf{x}) = \log P(C_i) + \sum_j \log p_{ij}(x_j)$$

- **Maximum likelihood estimator** for p_{ijk}:

$$\hat{p}_{ijk} = \frac{\text{number of times } x_j = v_k \text{ in class } C_i}{\text{total instances in class } C_i}$$

Multivariate Regression

- In **multivariate linear regression**, the numeric output r is assumed to be a **linear (weighted sum)** function of several input variables.
- In statistical literature, "multiple regression" = multiple inputs; "multivariate" = multiple outputs.

Multivariate Linear Model

$$\mathbf{r} = \mathbf{W}^T\mathbf{x} + \boldsymbol{\varepsilon}$$

- $\boldsymbol{\varepsilon}$ is assumed to be **normal** with mean zero and constant variance.
- Parameters are found by **maximizing (or minimizing) the sum of squared errors**:

$$E = \sum_t |\mathbf{r}^t - \mathbf{W}^T\mathbf{x}^t|^2$$

Dimensions of Supervised Machine Learning

Given training data: $\mathcal{X} = (\mathbf{x}^t, r^t)_{t=1}^N$

- $\mathbf{x}^t$ = arbitrary-dimensional input
- r^t = associated desired output (0 or 1 for binary classification)

To build a good approximation $g(\mathbf{x}^t|\theta)$ to r^t, three decisions must be made:

1. Model: $g(\mathbf{x}|\theta)$

- $g(\cdot)$ defines the **hypothesis class** $\mathcal{H}$
- θ = parameters; a particular value of θ instantiates one hypothesis $h \in \mathcal{H}$
- Example: Linear regression → $g(x|m,c) = mx + c$ where m, c are learned from data
- The model (inductive bias) is fixed by the **system designer**; the hypothesis is tuned by a **learning algorithm** using training data sampled from $p(\mathbf{x}, r)$

2. Loss Function: $L(\cdot)$

- Computes difference between desired output r^t and model output $g(\mathbf{x}^t|\theta)$
- **Total error** (approximation error):

$$E(\theta|\mathcal{X}) = \sum_{t=1}^{N} L(r^t,\ g(\mathbf{x}^t|\theta))$$

- Classification: $L(\cdot)$ checks equality → outputs 0 or 1
- Regression: $L(\cdot)$ = squared difference

3. Optimization Procedure

$$\theta^* = \arg\min_{\theta} \ E(\theta|\mathcal{X})$$

- In polynomial regression → **analytic solution** exists
- In general → **iterative methods** (e.g., gradient descent) are used
- Key concern: global minimum vs. local minima

For This to Work Well

1. Hypothesis class must have **enough capacity** to include the true function
2. Enough **training data** to pinpoint the correct hypothesis
3. A good **optimization method** to find the correct hypothesis

---

Parametric Classification

Posterior Probability

$$P(C_i|\mathbf{x}) = \frac{p(\mathbf{x}|C_i)P(C_i)}{p(\mathbf{x})}$$

Discriminant Function

$$g_i(\mathbf{x}) = \log p(\mathbf{x}|C_i) + \log P(C_i)$$

Gaussian Class-Conditional Density

$$p(\mathbf{x}|C_i) = \frac{1}{(2\pi)^{d/2}|\Sigma_i|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^T\Sigma_i^{-1}(\mathbf{x}-\boldsymbol{\mu}_i)\right)$$

Car Example

- $\mathbf{x}$ = customer's annual income
- $P(C_i)$ = proportion of customers buying car type i
- Each class has income distribution with mean μ_i and variance σ_i^2

Estimating Parameters

$$\hat{\mu}_i = \frac{1}{N_i}\sum_{t:r^t=C_i} \mathbf{x}^t, \quad \hat{P}(C_i) = \frac{N_i}{N}$$

Special Cases

When first term (constant) is neglected and priors are equal:

$$g_i(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \Sigma_i^{-1}(\mathbf{x} - \boldsymbol{\mu}_i) - \frac{1}{2}\log|\Sigma_i|$$

If variances are equal ($\Sigma_i = \Sigma$ for all i) → assign $\mathbf{x}$ to **nearest mean** class:

$$\hat{C} = \arg\min_i |\mathbf{x} - \boldsymbol{\mu}_i|^2$$

With two adjacent classes:

- Equal variances → **one threshold** (posteriors intersect at one point)
- Unequal variances → **two thresholds** (posteriors intersect at two points)

> 💡 When variances are unequal, a **reject region** with λ can be introduced to handle uncertain classifications.

Gradient Descent

Motivation

- In **likelihood-based classification**: parameters are sufficient statistics, estimated via MLE.
- In **discriminant-based approach**: parameters are those of discriminants, optimized to minimize classification error.
- Many cases have no analytical solution → use **iterative optimization** (gradient descent).

Gradient Vector

When $E(\mathbf{w})$ is a differentiable function of parameter vector $\mathbf{w}$:

$$\nabla E = \left[\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \cdots, \frac{\partial E}{\partial w_d}\right]^T$$

Gradient Descent Update Rule

$$\theta^{(k+1)} = \theta^{(k)} - \alpha \nabla J(\theta)$$

Where:

- θ = model parameters
- $J(\theta)$ = cost/loss function
- $\nabla J(\theta)$ = gradient of cost function
- α = learning rate (step size)
- k = iteration number

Key Points

- **Gradient Descent** → minimizes by going in **opposite direction** of gradient
- **Gradient Ascent** → maximizes by going in the **direction** of gradient
- When gradient = 0 → minimum (could be local or global)
- **No guarantee** of finding global minimum unless the function has only one minimum
- $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$ where η = learning rate

Worked Example

Given $J(w) = (w-3)^2$, $w_0 = 0$, $\alpha = 0.2$:

$$\frac{dJ}{dw} = 2(w-3)$$

Iteration	w	∇J	Update
0	0	$2(0-3) = -6$	$w_1 = 0 - 0.2(-6) = 1.2$
1	1.2	$2(1.2-3) = -3.6$	$w_2 = 1.2 - 0.2(-3.6) = 1.92$
2	1.92	$2(1.92-3) = -2.16$	$w_3 = 1.92 - 0.2(-2.16) = 2.352$

> Converges towards $w = 3$ (the minimum).

Logistic Discrimination

Definition

Logistic discrimination uses the **logistic (sigmoid) function** to model posterior class probabilities for classification. In modern ML, this is equivalent to **logistic regression** used for classification.

> Despite the name, logistic regression is a **classification algorithm**, not a regression one.

Mathematical Model

Component	Formula
Linear model	$z = \mathbf{w}^T\mathbf{x} + b$
Sigmoid function	$\sigma(z) = \dfrac{1}{1+e^{-z}}$
Output (probability)	$P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b)$

Decision Rule

$$\hat{y} = \{1, \quad P \geq 0.5\ 0, \quad \text{otherwise}$$

Two-Class Formulation

Using the **ratio of conditional densities**:

$$\log \frac{P(C_1|\mathbf{x})}{P(C_2|\mathbf{x})} = \mathbf{w}^T\mathbf{x} + w_0$$

Rearranging:

$$P(C_1|\mathbf{x}) = \frac{e^{\mathbf{w}^T\mathbf{x}+w_0}}{1+e^{\mathbf{w}^T\mathbf{x}+w_0}} = \sigma(\mathbf{w}^T\mathbf{x} + w_0)$$

This is the **sigmoid/logistic function** used as the estimator of $P(C_1|\mathbf{x})$.

Factors, Response and Strategy of Experimentation

Key Definitions

Term	Definition
Learner	Trained on a dataset; generates output for a given input
Experiment	A test (or series of tests) where factors affecting output are manipulated

Types of Factors

Controllable Factors — those we have control over:

- Learning algorithm used
- Hyperparameters (e.g., number of hidden units in MLP, k in KNN, C in SVM)
- Dataset and input representation/encoding

Uncontrollable Factors — those we cannot control (add undesired variability):

- Noise in the data
- Particular training subset (when resampling from large set)
- Randomness in optimization (e.g., initial state in gradient descent)

Response Variable (Output)

Used to measure quality of the experiment:

- Average classification error on test set
- Expected risk using a loss function
- Precision and recall
- Other performance metrics

Requirements of ML System

1. **Highest generalization accuracy** — performs well on unseen data
2. **Minimal complexity** — cheap in time and space
3. **Robustness** — minimally affected by external sources of variability

Strategies of Experimentation

Given 2 factors (F) and 5 levels (L) each:

Strategy	Description	Time Complexity
Best Guess	Start with a guess; modify factors non-systematically; no guarantee of best config	—
One Factor at a Time	Fix baseline for all factors; change one at a time	$O(L \cdot F)$
Factorial Design (Grid Search)	Vary all factors together	$O(L^F)$

Response Surface Design

- **Fit a parametric response function** to the factors to reduce the number of searches.

$$r = g(f_1, f_2, \ldots, f_F \mid \phi)$$

Where r is the response and f_i are the factors.

Methods

1. **Fractional factorial design** — run only a subset of all possible combinations.
2. **Use knowledge from previous runs** to estimate configurations likely to have high response.

Procedure

1. Assume $g(\cdot)$ is a (typically **quadratic**) regression model.
2. Run a small number of experiments around some baseline → fit $g(\cdot)$.

3. **Analytically calculate** the values of f_i where fitted g is maximum → new guess.
4. Run experiment there → add data instance → refit g → repeat until convergence.

> ⚠️ Success depends on whether the response can actually be written as a quadratic function with a **single maximum**.

Randomization, Replication and Blocking

These are the **three basic principles** of experimental design.

1. Randomization

- The order of runs should be **randomly determined** so results are independent.
- Removes **bias** — ensures each model training run has an equal chance of receiving different data samples, preventing confounding factors from skewing results.
- Tests done in random order so as to not bias results.

2. Replication

- For the same configuration of controllable factors, the experiment must be **run multiple times**.
- Purpose: **average over the effect** of undesired uncontrollable factors (on resampled versions of same dataset).
- Example: **Cross-validation** — the variation in response across multiple experiments allows estimation of experimental error.
- Ensures **repeatability**.

3. Blocking

- Reduces or eliminates variability due to **nuisance factors** — variables that influence the response but are not of primary interest.
- Example: defects in a factory may depend on different batches of raw material.
- In ML: when comparing models, **use the same resampled subsets of data** — otherwise accuracies will depend on different subsets.
- Goal: **measure difference due to algorithms only**.

> 💡 Nuisance factor = a variable that influences the outcome but is not what we're testing.

Guidelines for Machine Learning Experiments

A. Aim of the Study

- State the problem clearly and define objectives.
- We may be interested in:
  - Expected error
  - Determining which algorithm has less generalization error
  - Comparing two or more algorithms for the least error
  - Comparing algorithms on different datasets

B. Selection of Response Variable

- Decide on the quality measure: Error, Precision & Recall, Complexity, etc.

C. Choice of Factors and Levels

- Factors depend on the aim of the study (e.g., when comparing algorithms, the algorithm is a factor).
- Levels must be selected to not miss a good configuration and avoid unnecessary experimentation.
- Investigate **all factors and factor levels** — don't be biased by previous experience.

D. Choice of Experimental Design

- Factorial design is generally preferred.
- Replication depends on dataset size — too few replicates make comparing distributions difficult.
- Division of dataset into **training, validation, and testing** is important (small datasets → high variance response).
- Prefer **real data** over synthetic data.

E. Performing Experiments

- Do a few trial runs for random settings.
- **Save intermediate results** so experiments can be rerun.
- All results should be **reproducible**.
- Be aware of software aging effects.
- Experimenter should be **unbiased**.
- Use code from **reliable sources**.
- Good **documentation** is essential.

F. Statistical Analysis of Data

- Analyze data so conclusions are not subjective or by chance.
- Cast questions in the framework of **hypothesis testing**.
- **Visual analysis** is always more helpful.

G. Conclusions and Recommendations

- First runs are for **investigation only** — do not start with high expectations.
- Statistical testing never tells if a hypothesis is correct or false, only how much the sample concurs with it.
- When expectations are not met, **investigate why** — finding a deficiency helps improve the system.
- Do not move to the next step without completely analyzing current data.

Cross-Validation and Resampling Methods

Purpose

- For replication, we need training/validation set pairs from dataset $\mathcal{X}$.
- If $\mathcal{X}$ is large: randomly divide into K parts, then split each into training/validation halves.
- For **small datasets**: use **cross-validation**.
- Goal: generate K training/validation pairs $T_i, V_i{}_{i=1}^{K}$ with:
  - Large number of pairs
  - Minimal overlap between sets
  - **Stratification** — maintain class proportions in each subset

K-Fold Cross-Validation

1. Divide dataset $\mathcal{X}$ into K equal-sized parts $X_i,\ i = 1, \ldots, K$.
2. For each fold: one part = **validation set**; remaining $K - 1$ parts = **training set**.
3. Train and evaluate K times; average the results.

```
Example (K=3): Data = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
Fold1: [0.5, 0.2] | Fold2: [0.1, 0.3] | Fold3: [0.4, 0.6]

Model1: Train on Fold1+Fold2, Test on Fold3
Model2: Train on Fold2+Fold3, Test on Fold1
Model3: Train on Fold1+Fold3, Test on Fold2
```

Trade-offs:

K	**Training Set Size**	**Validation Set Size**	**Overlap**	**Computation**
Large	Large (robust estimators)	Small	High	High
Small	Small	Large	Low	Low

> Typical values: $K \sim 10$ or $K \sim 30$

Leave-One-Out (LOO) Cross-Validation

- Special case of K-fold where $K = N$.
- For each of N instances, leave one out as validation; train on remaining $N - 1$.
- Maximum use of data, but computationally expensive.

5×2 Cross-Validation

1. **Fold 1**: Randomly split $\mathcal{X}$ into halves $X_1^{(1)}$ and $X_1^{(2)}$.
   - Pair 1: $T_1 = X_1^{(1)}$, $V_1 = X_1^{(2)}$
   - Pair 2 (swap): $T_2 = X_1^{(2)}$, $V_2 = X_1^{(1)}$
2. Shuffle $\mathcal{X}$ and repeat for 4 more folds → **10 training/validation sets** total.

Why 5 folds?

- More than 5: sets share many instances → estimates too dependent, add no new information.
- Fewer than 5: fewer than 10 sets → insufficient data to test hypothesis.

Bootstrap

- Generate new samples by **drawing instances from original sample with replacement**.
- Bootstrap samples may overlap more than CV samples → estimates are more dependent.
- Considered **best for very small datasets**.

Key statistic:

$$P(\text{instance not picked after } N \text{ draws}) = \left(1 - \frac{1}{N}\right)^N \approx e^{-1} \approx 0.368$$

- Training data contains approximately **63.2%** of instances.
- **36.8%** are not seen during training → error estimate is **pessimistic**.
- Unseen instances = **out-of-bag (OOB)** samples → used for validation.

```
Example: Data = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6], sample size = 4
Bootstrap sample: [0.2, 0.1, 0.2, 0.6]   ← with replacement
OOB (validation): [0.3, 0.4, 0.5]
```

Contour Plots of Gaussian Distributions

Multivariate Gaussian

$$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu},\ \boldsymbol{\Sigma})$$

For a 2D Gaussian (X_1, X_2), the density is:

$$f(\mathbf{x}) = \frac{1}{2\pi|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

Contour Plots

A contour plot fixes the density to a constant value:

$$(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) = c$$

This defines an **ellipse**. Each contour:

- Encloses points with the **same probability density**
- Corresponds to a constant **Mahalanobis distance** from the mean

Properties

Property	**Details**
Center	All contours centered at $\boldsymbol{\mu}$
Shape	Determined by $\boldsymbol{\Sigma}$
Orientation	Given by eigenvectors of $\boldsymbol{\Sigma}$
Axis length	Given by eigenvalues of $\boldsymbol{\Sigma}$

Special Cases

$\boldsymbol{\Sigma}$	**Contour Shape**
$\sigma^2\mathbf{I}$	**Circular** (isotropic)
Correlated variables	**Tilted ellipses**
Different variances, no correlation	**Axis-aligned stretched ellipses**

Interval Estimation

Point vs. Interval Estimation

- **Point estimator** (e.g., MLE): specifies a single value for parameter θ.
- **Interval estimation**: specifies an **interval** within which θ lies with a certain degree of confidence.

Example: Estimating Mean μ

Given sample $\mathcal{X} = x^t{}_{t=1}^N$ from $\mathcal{N}(\mu, \sigma^2)$:

$$m = \frac{1}{N}\sum_t x^t \quad \text{(sample mean, point estimator)}$$

Since m is a sum of normals: $m \sim \mathcal{N}\left(\mu,\ \frac{\sigma^2}{N}\right)$

The standardized statistic:

$$Z = \frac{m - \mu}{\sigma/\sqrt{N}} \sim \mathcal{N}(0,1)$$

Confidence Interval Steps

1. Gather sample data and calculate sample mean $\bar{x}$.
2. Determine if population standard deviation σ is known or unknown.
3. Use appropriate statistic:
   - **Known** σ: use z-score
   - **Unknown** σ: use t-statistic

$$\text{Confidence Interval} = \bar{x} \pm \text{margin of error}$$

Two-Sided Confidence Interval (Known σ)

Since $P\{-1.96 < Z < 1.96\} = 0.95$:

$$P\left\{m - 1.96\frac{\sigma}{\sqrt{N}} < \mu < m + 1.96\frac{\sigma}{\sqrt{N}}\right\} = 0.95$$

For general confidence level $1-\alpha$, define z_α such that $P\{Z > z_\alpha\} = \alpha$:

$$P\{-z_{\alpha/2} < Z < z_{\alpha/2}\} = 1 - \alpha$$

So a $100(1-\alpha)$ confidence interval for μ is:

$$\left(m - z_{\alpha/2}\frac{\sigma}{\sqrt{N}},\quad m + z_{\alpha/2}\frac{\sigma}{\sqrt{N}}\right)$$

> 💡 As N (sample size) increases → interval gets **narrower** → more precision.
> As confidence level increases → interval gets **wider**.

One-Sided Confidence Interval

For 95% one-sided (lower bound on μ):

$$PZ < 1.64 = 0.95 \implies m - 1.64\frac{\sigma}{\sqrt{N}} \text{ is the lower bound}$$

Unknown σ: t-Distribution

When σ is unknown, use the **sample variance** S^2. The statistic:

$$T = \frac{m - \mu}{S/\sqrt{N}} \sim t_{N-1} \quad (\text{t-distribution with } N-1 \text{ d.o.f.})$$

This uses the fact that $(N-1)S^2/\sigma^2 \sim \chi^2_{N-1}$.

Confidence interval using t:

$$\left(m - t_{\alpha/2,\, N-1} \cdot \frac{S}{\sqrt{N}}, \quad m + t_{\alpha/2,\, N-1} \cdot \frac{S}{\sqrt{N}} \right)$$

> ⚠️ t-distribution has **longer tails** than standard normal → wider intervals (reflects additional uncertainty from unknown σ).

Hypothesis Testing

Concept

- Instead of estimating a parameter, we want to **test a specific hypothesis** about it.
- If the sample is **consistent** with the hypothesis → **fail to reject**.
- Otherwise → **reject**.

Procedure: Define a statistic that obeys a known distribution **if the hypothesis is correct**. If the calculated statistic has very low probability under that distribution → **reject** the hypothesis.

Null and Alternative Hypothesis

Given sample from $\mathcal{N}(\mu, \sigma^2)$ (with σ^2 known), testing:

$$H_0 : \mu = \mu_0 \quad \text{vs} \quad H_1 : \mu \neq \mu_0$$

- m = point estimate of μ
- Reasonable to **reject** H_0 if m is too far from μ_0

Decision Rule (Two-Sided Test, Known σ)

Fail to reject H_0 at significance level α if μ_0 lies in the $100(1-\alpha)$ confidence interval:

$$|m - \mu_0| \leq z_{\alpha/2} \cdot \frac{\sigma}{\sqrt{N}}$$

Reject H_0 if it falls outside.

Types of Errors

Error	Definition	Probability
Type I Error	Reject H_0 when it is actually correct	α (significance level)
Type II Error	Fail to reject H_0 when it is actually false	$\beta(\mu)$

- Typical values: $\alpha = 0.1,\ 0.05,\ 0.01$
- **Power function**: $1 - \beta(\mu)$ = probability of rejection when μ is the true value
- Type II error increases as μ gets closer to μ_0

One-Sided Test

$$H_0 : \mu \leq \mu_0 \quad \text{vs} \quad H_1 : \mu > \mu_0$$

Fail to reject if:

$$m \leq \mu_0 + z_\alpha \cdot \frac{\sigma}{\sqrt{N}}$$

Unknown σ: Two-Sided t-Test

$$H_0 : \mu = \mu_0 \quad \text{vs} \quad H_1 : \mu \neq \mu_0$$

Fail to reject at significance level α if:

$$\left| \frac{m - \mu_0}{S/\sqrt{N}} \right| \leq t_{\alpha/2,\, N-1}$$

where $t_{\alpha/2,\, N-1}$ is the critical value from the t-distribution with $N - 1$ degrees of freedom.

Z-Table Reference

Negative Z Table → z-values to the **left** of the mean
Positive Z Table → z-values to the **right** of the mean

Common critical values:

Confidence	α	$z_{\alpha/2}$
90%	0.10	1.645
95%	0.05	1.960
99%	0.01	2.576
